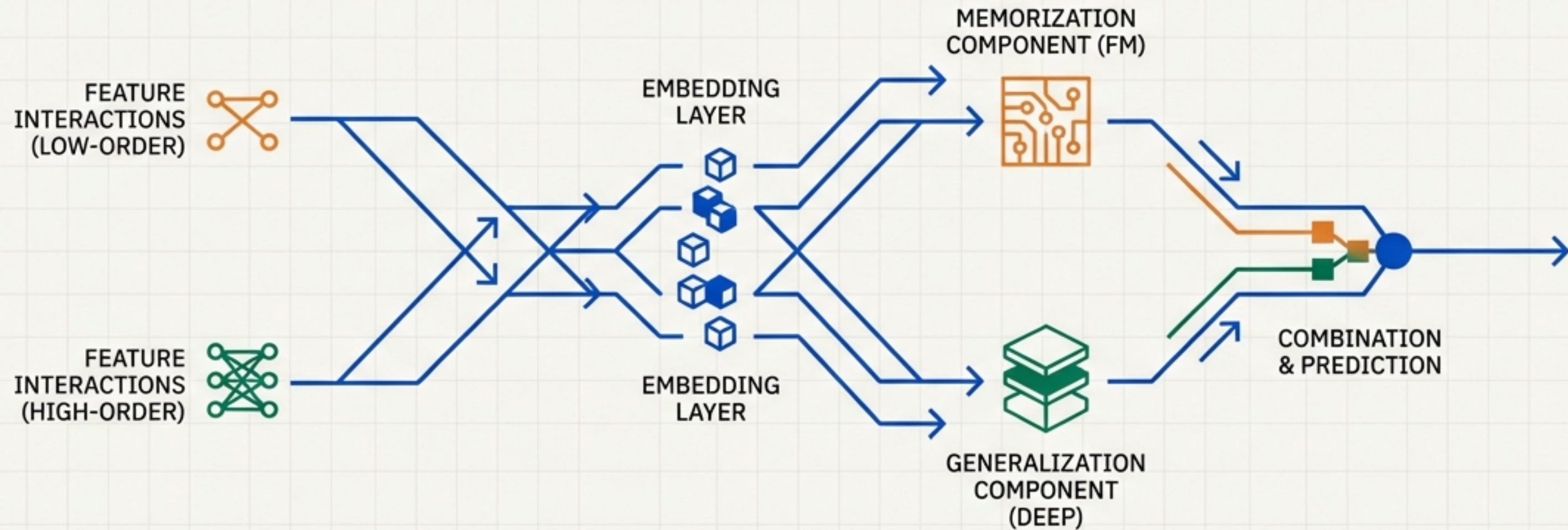


Reimplementing DeepFM: A Practical Guide to CTR Prediction

An end-to-end model implementation for learning low and high-order feature interactions.



Explore the Project:

``https://github.com/RuokeZhang/CTR_Prediction``

The Foundation: The DeepFM Paper and Criteo Dataset



DeepFM: A Factorization-Machine based Neural Network for CTR Prediction

Authors: Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, Xiuqiang He

Publication: IJCAI 2017

Core Idea: An end-to-end learning model that combines the power of factorization machines for recommendation and deep learning for feature learning.



Criteo Display Ad Challenge

Source: Criteo AI Lab

Scale: ~45 million user click records.

Features: A challenging mix of 13 numerical (dense) features and 26 categorical (sparse) features.

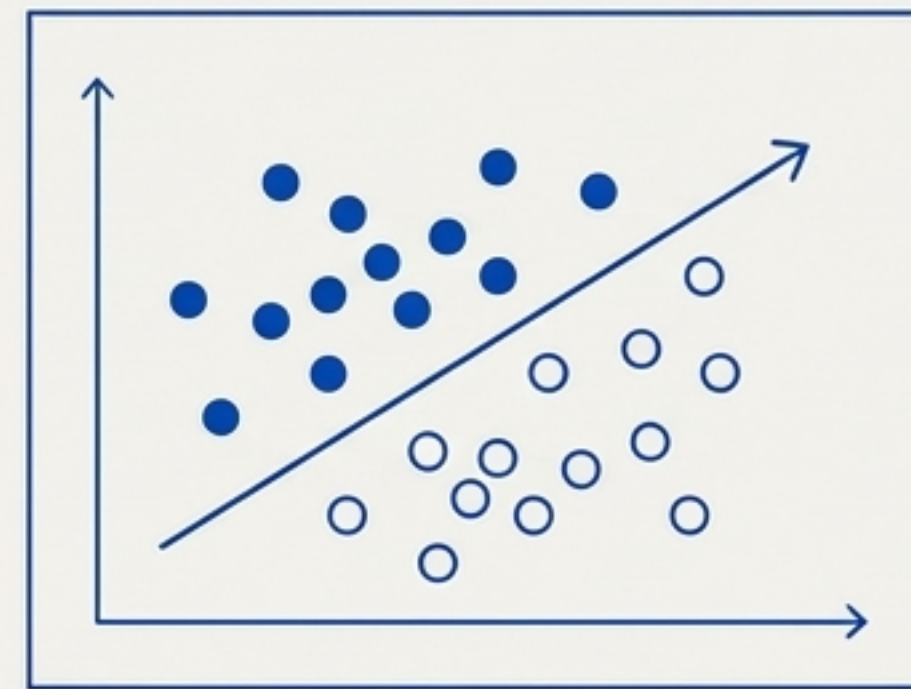
Task: Binary classification - predict whether a displayed ad was clicked (1) or not (0) or not (0).

Establishing the Baseline and Defining Success

The Baseline Model

Logistic Regression (LR)

A simple, widely-used linear model. It learns the importance of individual features but cannot capture non-linear feature interactions without manual feature engineering. Serves as a strong, efficient baseline.



Key Evaluation Metrics

AUC (Area Under the ROC Curve)

Measures the quality of the model's ranking. Represents the probability that a randomly chosen positive sample is ranked higher than a randomly chosen negative sample. Higher is better.



Logloss (Binary Cross-Entropy)

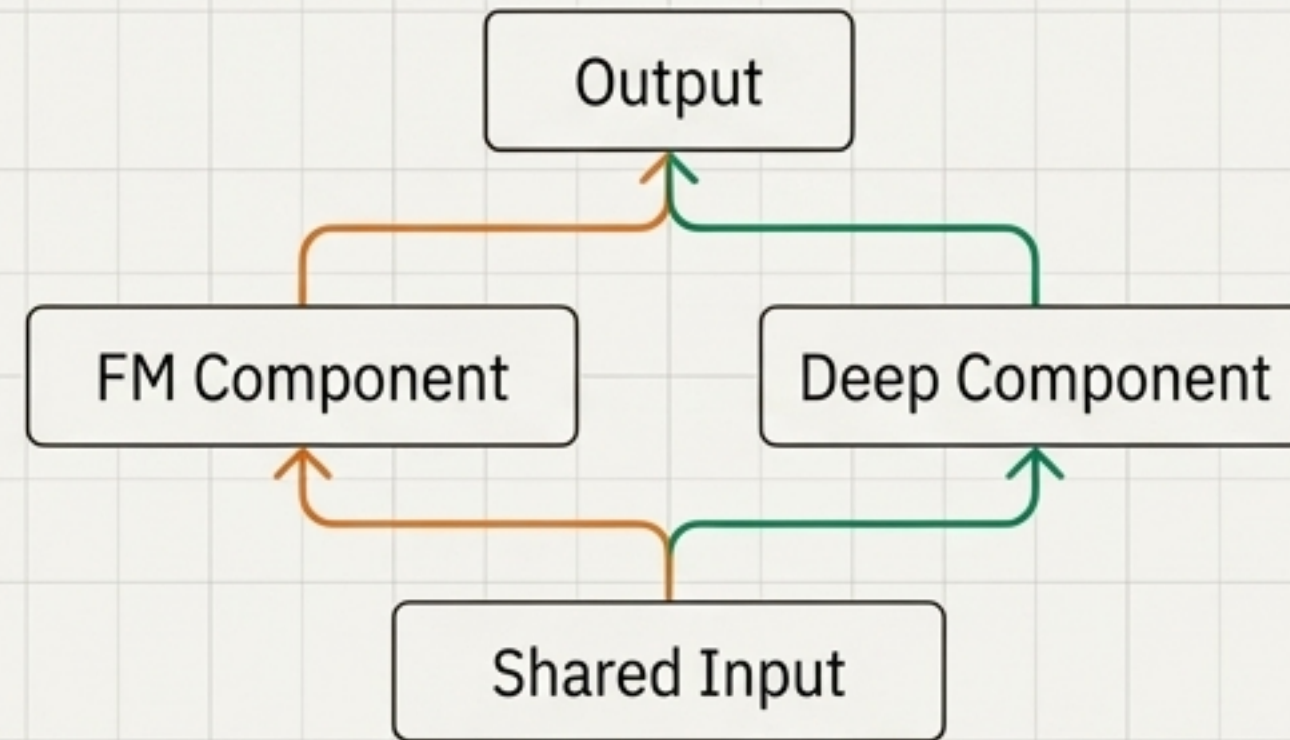
Measures the accuracy of the predicted probabilities. It penalizes predictions that are confident and wrong. Lower is better.



$$L = - (y * \log(p) + (1 - y) * \log(1 - p))$$

The DeepFM Architecture: Combining Memorization and Generalization

DeepFM is a unified, end-to-end network that learns feature interactions of all orders from raw features. It achieves this with two parallel components that share the same input.



Memorization Engine

Excels at learning simple, direct, and frequently seen feature interactions (e.g., “users in New York are more likely to click on taxi ads”). Captures low-order (1st and 2nd) interactions efficiently.

Generalization Engine

Excels at discovering complex, subtle, and unseen feature interactions (e.g., “users who like three unrelated product categories might also be interested in this new ad”). Captures high-order interactions via a deep neural network.

Step 1: The Foundation of Shared Embeddings

Problem:

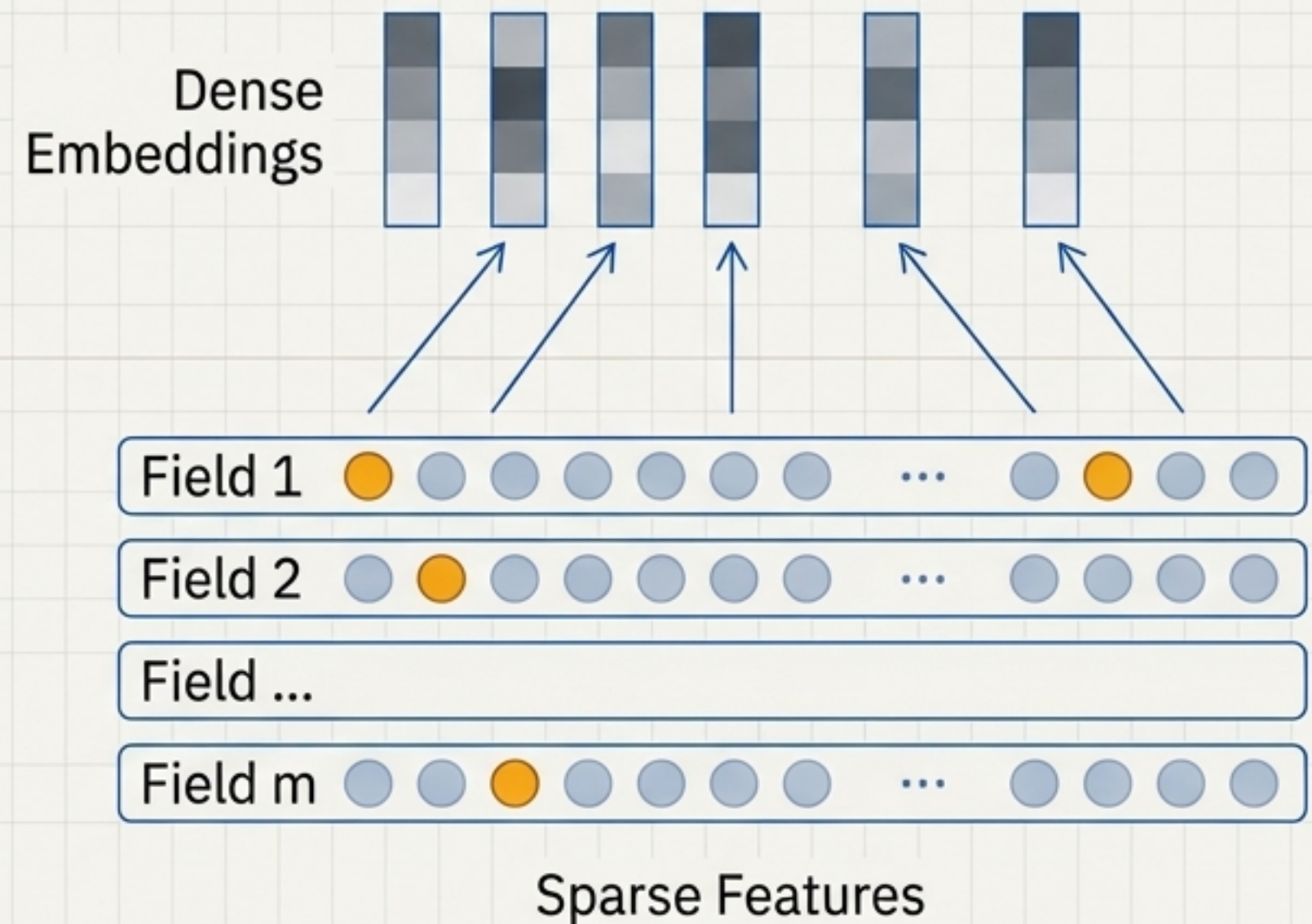
CTR data is high-dimensional and extremely sparse. A one-hot encoded vector would be massive and inefficient.

Solution: An Embedding Layer

Each high-cardinality categorical feature is mapped to a low-dimensional, dense vector (an 'embedding'). In this project, the embedding size is 8.

Key Insight:

This dense representation captures latent relationships. This same embedding layer serves as the input for *both* the FM and Deep components.



Step 2: Building the 'Memorization' Engine (The FM Component)

Goal: To model low-order feature interactions directly and efficiently.

Part A: 1st-Order Interactions (The Linear Part)

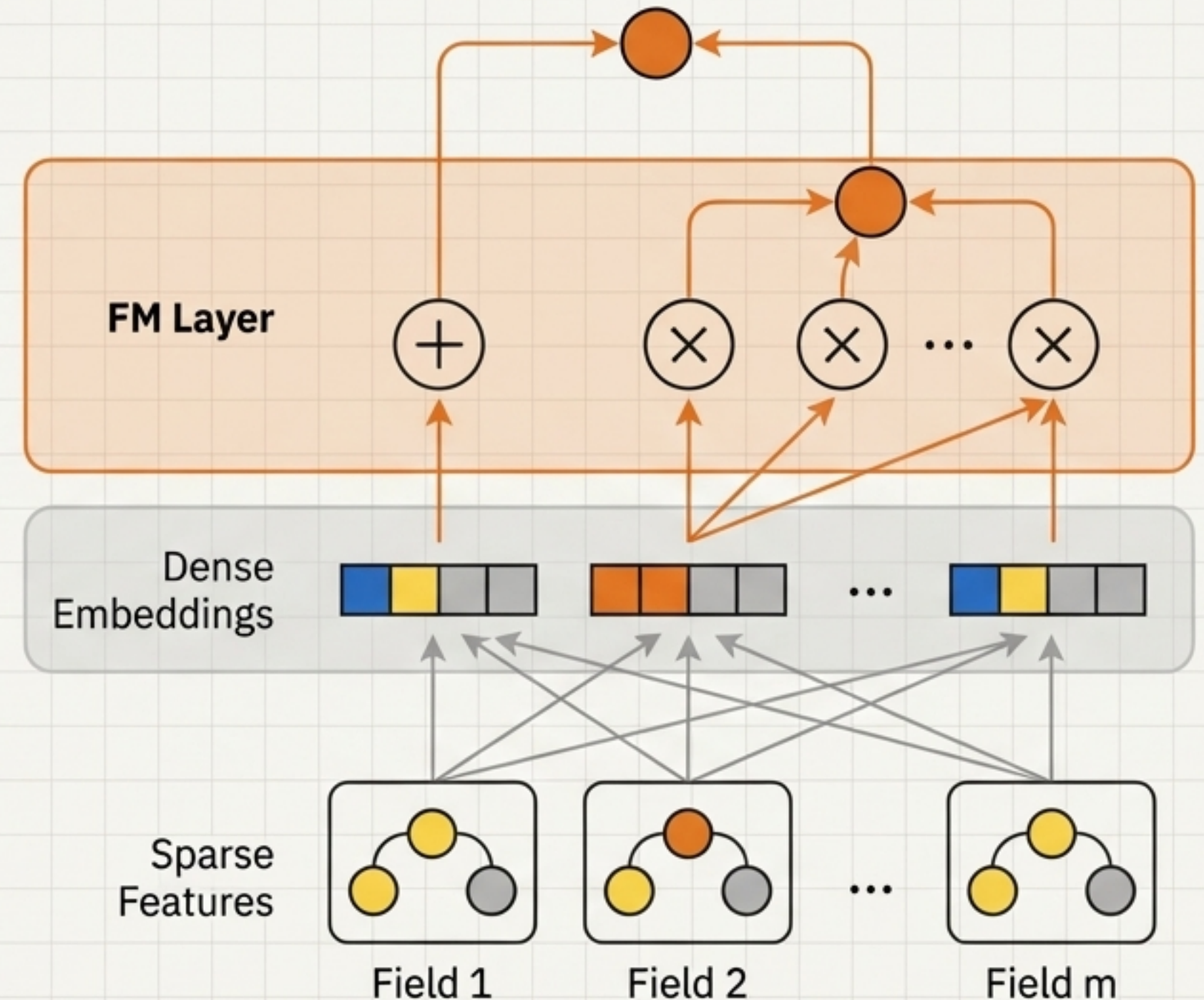
A simple addition of weighted features, exactly like in Logistic Regression. Captures the direct importance of each individual feature.

$$y_{\text{linear}} = \langle \mathbf{w}, \mathbf{x} \rangle$$

Part B: 2nd-Order Interactions (The FM Part)

Models the interaction between pairs of features by taking the inner product of their respective embedding vectors. A highly efficient way to learn pairwise interactions.

$$y_{\text{pairwise}} = \sum \langle V_i, V_j \rangle x_i * x_j$$



Step 3: Building the “Generalization” Engine (The Deep Component)

Goal: To learn high-order, non-linear, and implicit feature interactions.

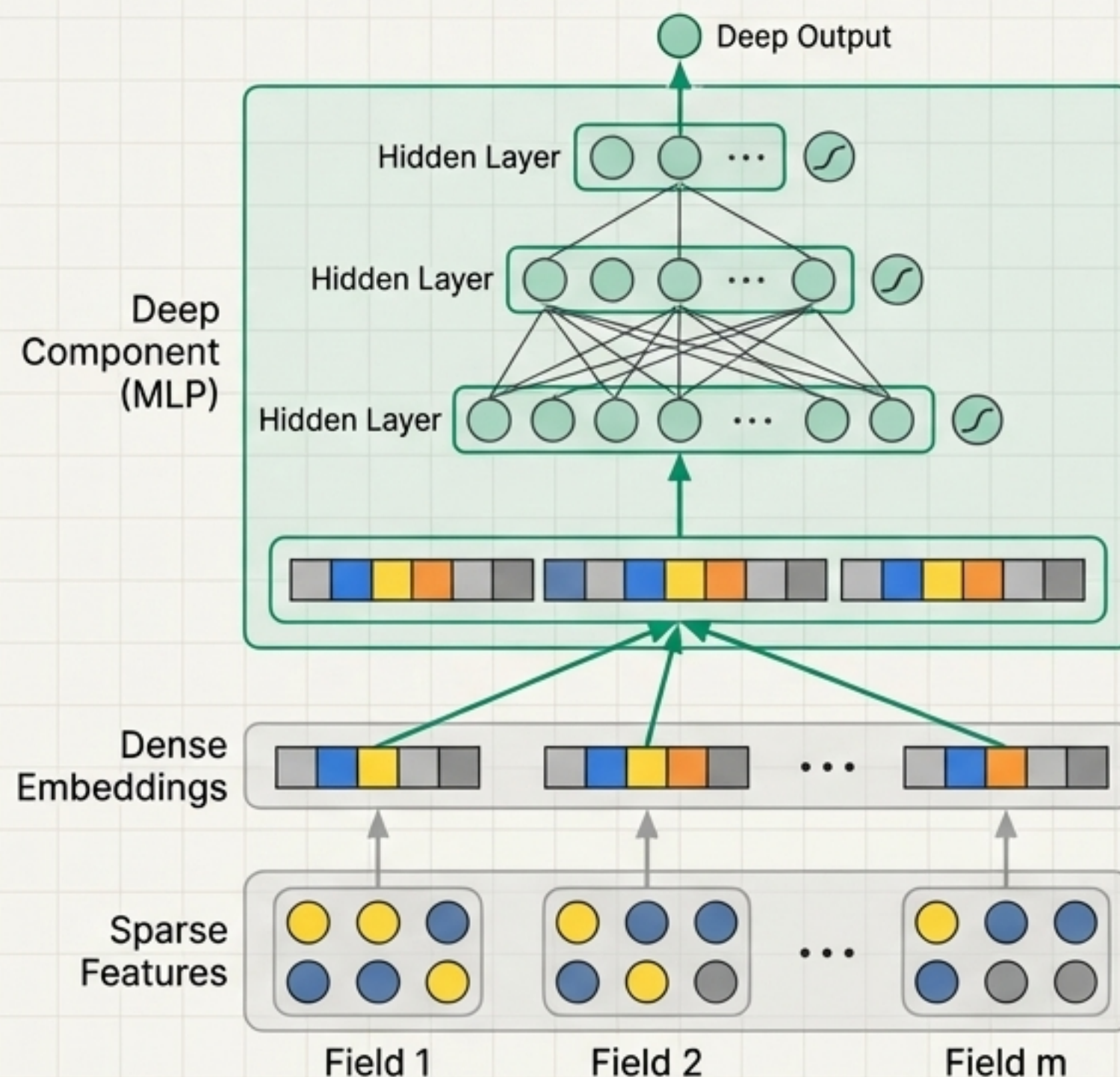
Architecture: A standard feed-forward neural network (MLP).

1. The dense embedding vectors for all features are concatenated into a single, long vector.
2. This vector is fed through a series of hidden layers with non-linear activation functions.
3. The network learns complex combinations and abstractions of the input features.

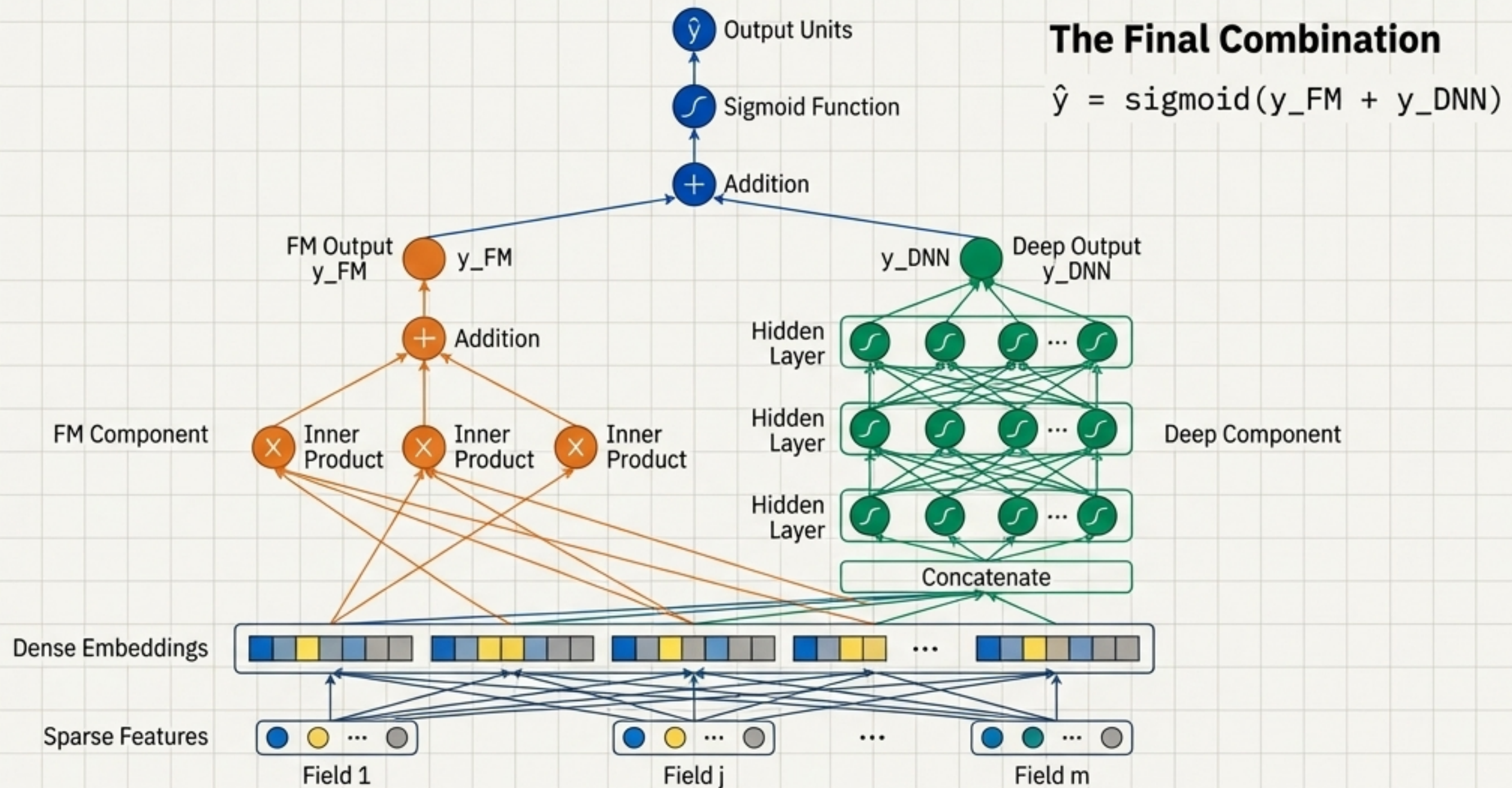
Project Specific Architecture:

Hidden Layers: 3

Hidden Dimensions: [256, 128, 64]



Step 4: The Synergy of DeepFM



Key Advantages

- 1. End-to-End Learning:** No pre-training or manual feature engineering required. Learns directly from raw features.
- 2. Shared Embeddings:** The embedding layer is learned jointly, leading to richer and more expressive feature representations.

The Results: DeepFM Delivers a Significant Performance Lift

Experiment Setup

Dataset: Criteo

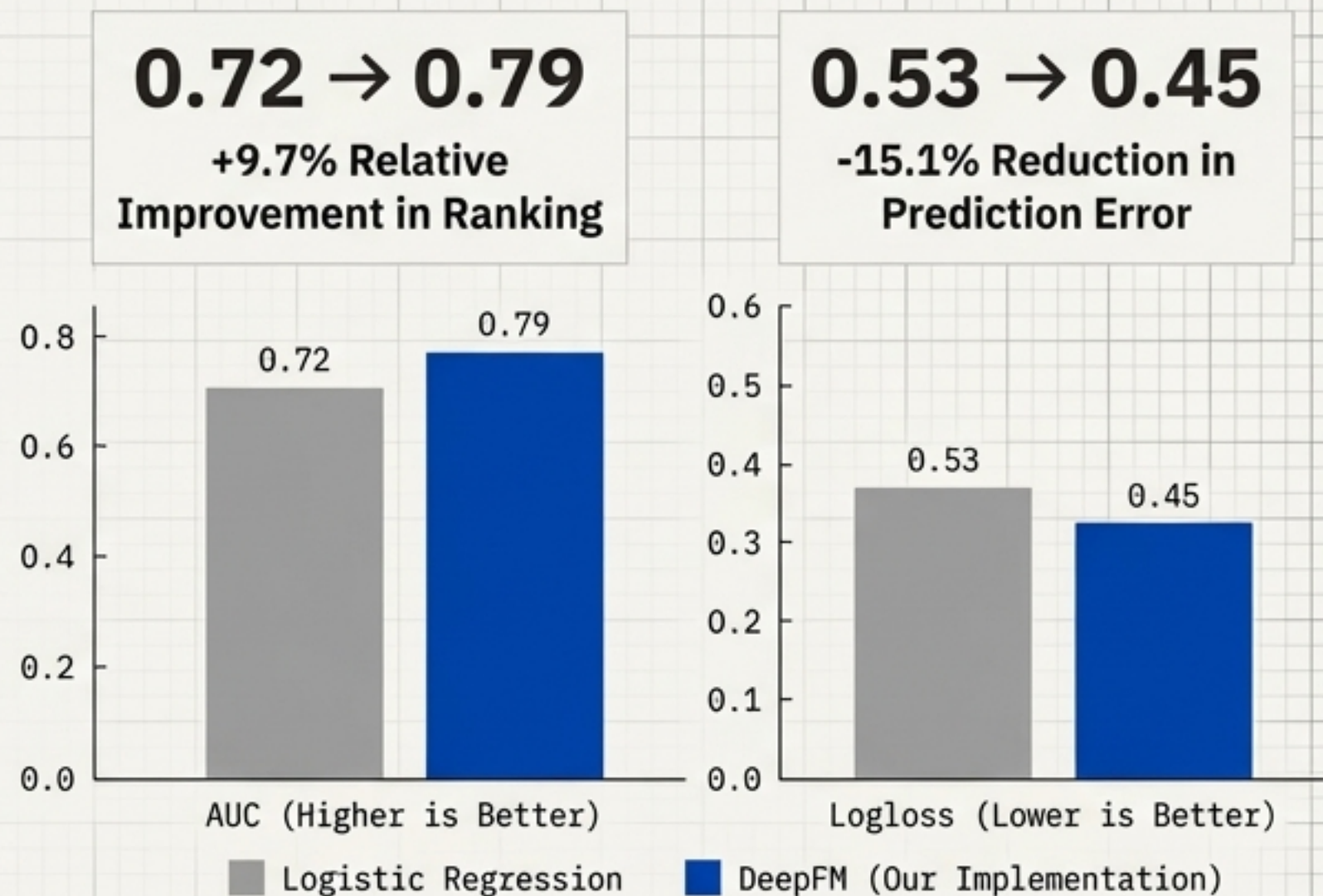
Baseline: Logistic Regression (LR)

Epochs: 15

Embedding Size: 8

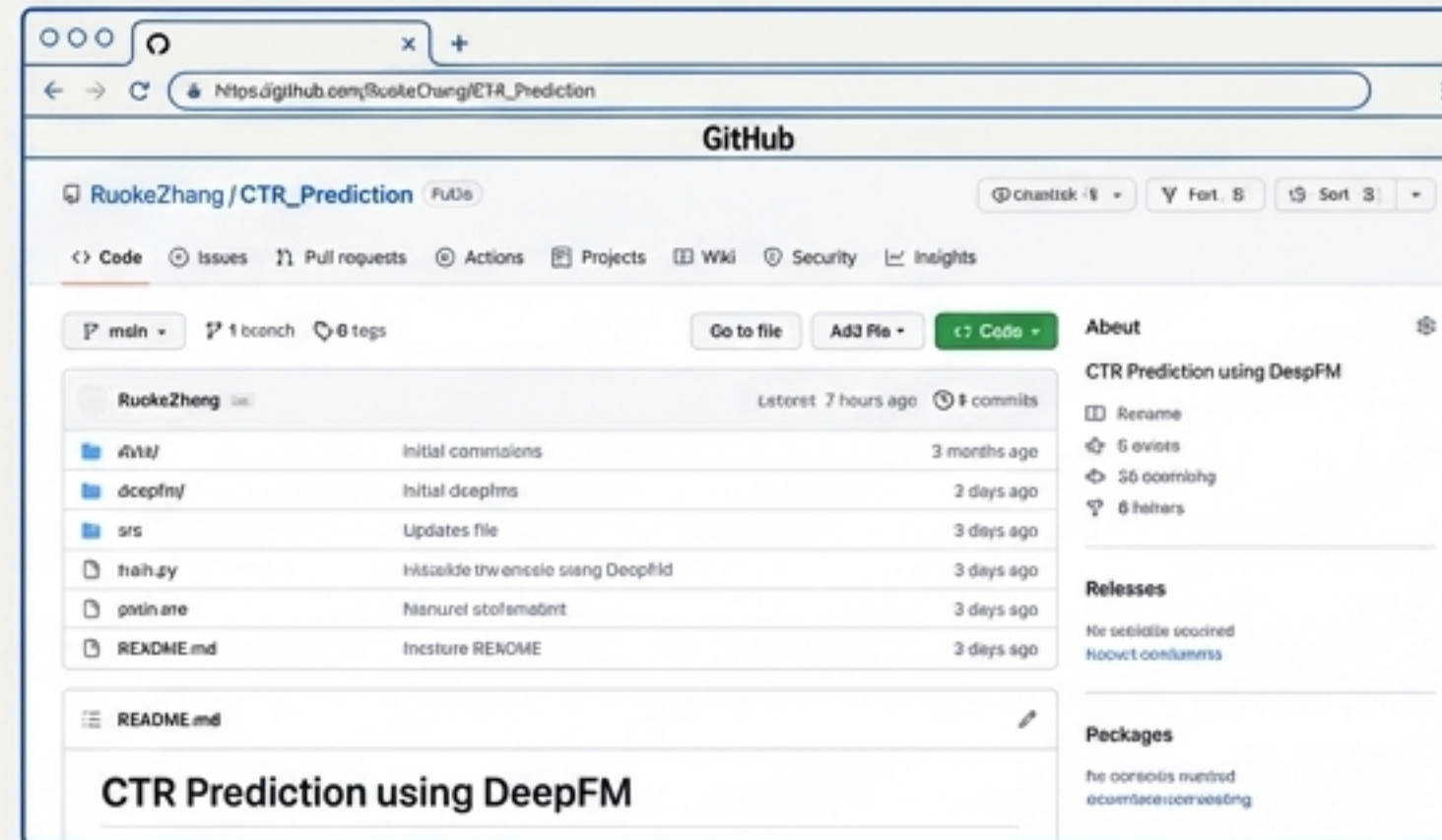
Hidden Dims: [256, 128, 64]

Performance Comparison



An Elegant Architecture for a Complex Problem

DeepFM's power lies in its unified structure. By sharing feature embeddings, it efficiently learns both the simple, memorable patterns and the complex, generalizable rules needed for accurate CTR prediction.



Dive Deeper and Build Your Own

The complete, commented code is available for you to explore, run, and adapt.

https://github.com/RuokeZhang/CTR_Prediction